Karnaugh Map

2023/10/23

B113040045 許育菖

實驗內容

Exercise1

Let F = (AB' +A'B)(C+ D')

- Implement F with **AND-OR gates** (denoted as F1)
- Implement F with multi-level **NAND gate** circuit (denoted as F2)
- Implement F with multi-level **NOR gate** circuit (denoted as F3)

Verify F1 = F2 = F3 (using **Verilog Structural level modeling**)

使用 DeMorgan’s theorem 反運算:

F1 = **AB'C + A'BC + AB'D + A'BD**

⇒ F2 = **[(AB'C)' (A'BC)' (AB'D)' (A'BD)']'** *(SOP)*

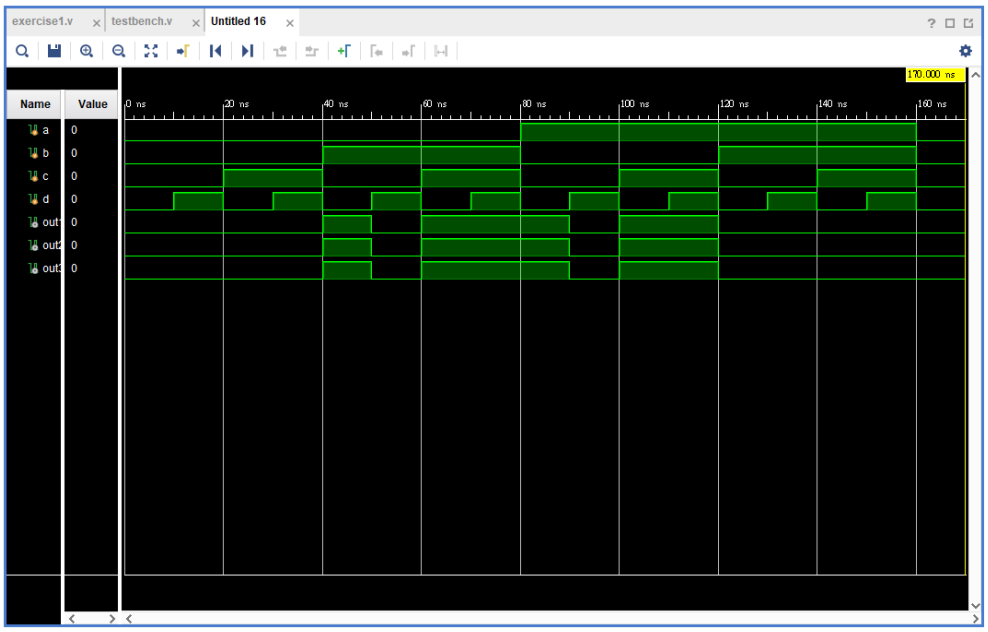
F3 = (A' + B') (C + D) (A + B) (SOP)

= **[(A' + B')' + (C + D)' + (A + B)']'** *(POS)*

Verilog code:

```
`timescale 1ns / 1ps

module exercise1(A, B, C, D, F1, F2, F3);
    output F1, F2, F3;
    input A, B, C, D;
//  F1
```

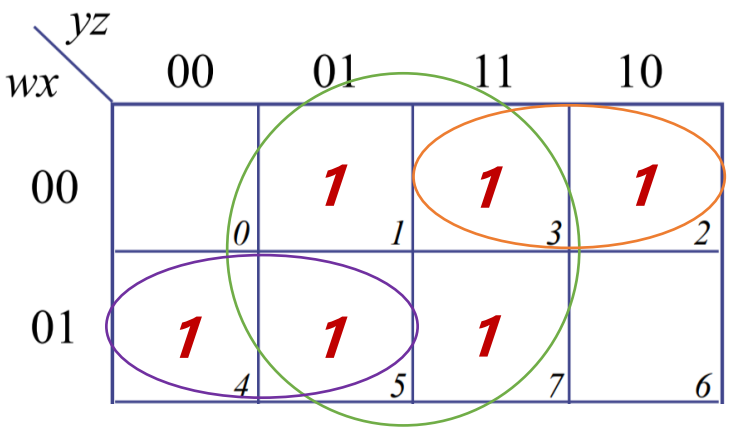
```
wire and1, and2, and3, and4;
and(and1, A, !B, C);
and(and2, !A, B, C);
and(and3, A, !B, !D);
and(and4, !A, B, !D);
or(F1, and1, and2, and3, and4);

//   F2
wire and5;
and(and5, !and1, !and2, !and3, !and4);
not(F2, and5);

//   F3
wire or1, or2, or3, or4;
or(or1, !A, !B);
or(or2, A, B);
or(or3, C, !D);

or(or4, !or1, !or2, !or3);
not(F3, or4);
endmodule
```



從實驗結果可發現: F = F1 = F2 = F3

Exercise2

1. Simplify the following Boolean function using **Karnaugh maps**: F(x, y, z) = S (1, 2, 3, 4, 5, 7).
2. Find the simplest sum of products (F4) and draw its logic diagram (two-level implementation).
3. Implement F4 with **two-level NAND gate** circuit (denoted as F5).
4. F6 = (F4')' , implement F6 with **two-level OR-NAND** (OAI) circuit.
5. Verify F4 = F5 = F6 (using **Verilog Structural level modeling**)



(w 沒用到)

F4 = **z + x'y + xy'**

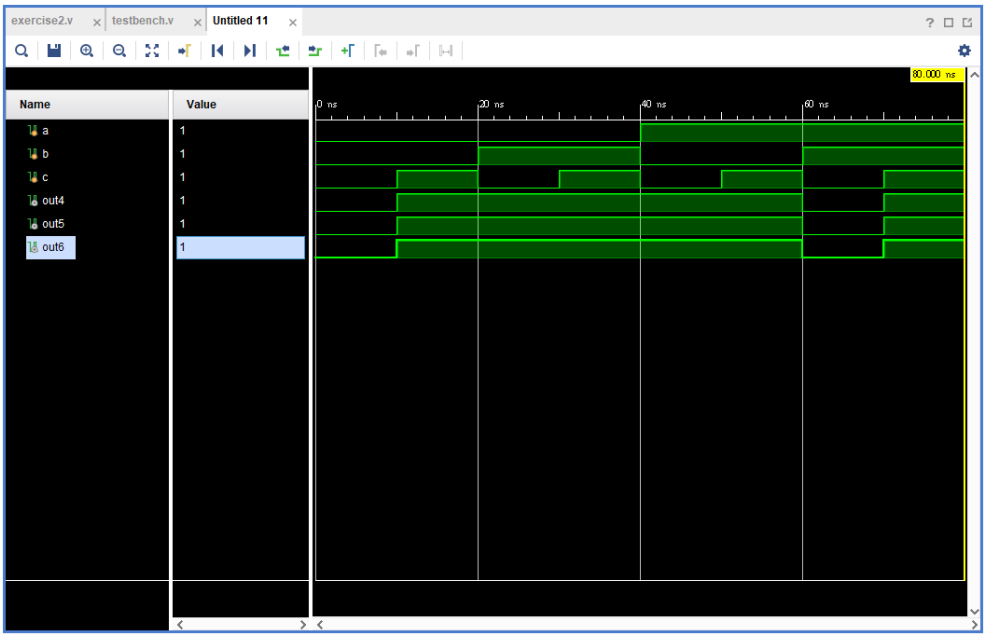
F5 = **[z' (x'y)' (xy')']'**

F6 = **[z' (x + y') (x' + y)]'**

Verilog code

```
`timescale 1ns / 1ps

module exercise2(x, y, z, F4, F5, F6);
    output F4, F5, F6;
    input x, y, z;
```

```
    //  F4
    wire and1, and2;
    and(and1, x, !y);
    and(and2, !x, y);

    or(F4, and1, and2, z);

    //  F5
    wire and4;
    and(and4, !and1, !and2, !z);
    not(F5, and4);

    //  F6
    wire or1, or2, and5;
    or(or1, !x, !y, z);
    or(or2, x, y, z);
    nand(and5, or1, or2);
    not(F6, and5);
    //  not(F6, and5);
endmodule
```



從實驗結果可以發現，F4 = F5 = F6。

Exercise3

1. Simplify the following Boolean function using Karnaugh maps:

   F(x, y, z) = S (1, 2, 3, 4, 5, 7)

2. Find the simplest **product of sums** (F7) and draw its logic diagram (two-level implementation)

3. Implement F7 with two-level **NOR gate** circuit (denoted as F8)

4. Implement F7 with two-level **AND-NOR (AOI)** circuit (denoted as F9)

5. Verify F7 = F8 = F9 (using **Verilog Structural level modeling**)

---

wx \ yz	00	01	11	10
00	0	1 (1)	1 (3)	1 (2)
01	1 (4)	1 (5)	1 (7)	6

(w 沒用到)

F7 = **(x + y + z)(x' + y' + z)**

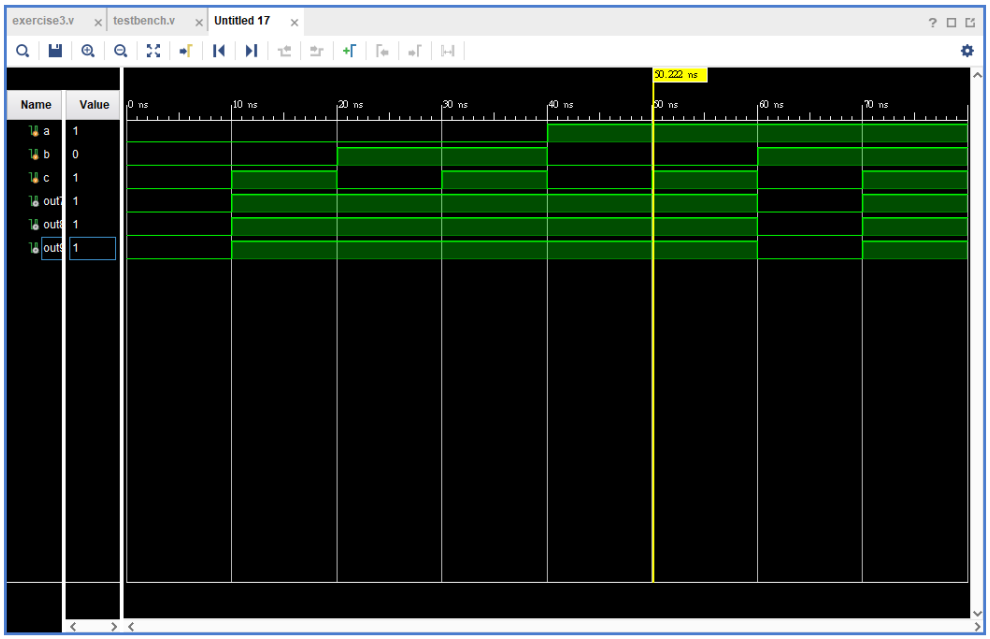
⇒ F8 = **[(x + y + z)' + (x' + y' + z)']'**

F9 = **[(x'y'z') + (xyz')]'**

Verilog code

```
`timescale 1ns / 1ps

module exercise3(x, y, z, F7, F8, F9);
```

```
    output F7, F8, F9;
    input x, y, z;

    //   F7
    wire or1, or2;
    or(or1, x, y, z);
    or(or2, !x, !y, z);
    and(F7, or1, or2);

    //   F8
    wire nor1, nor2;
    nor(nor1, x, y, z);
    nor(nor2, !x, !y, z);
    nor(F8, nor1, nor2);

    //   F9
    wire and1, and2;
    and(and1, !x, !y, !z);
    and(and2, x, y, !z);
    nor(F9, and1, and2);

endmodule
```



從實驗結果可以證明：F7 = F8 = F9

實驗心得

還滿感謝教授讓我們進行這次的練習，因為在這次實作之前，我對於 POS 和 SOP 之間的差別一直很不清楚，我一直以為 POS 和 SOP 都一樣是尋找 Kar-naugh Map 中值為 true 的格子，只是其中 SOP 是**先乘再加**，而 POS 是**先加再乘**而以。這次實作後才搞清楚，原來 SOP 是尋找值為 true 的格子，POS 則是尋找值為 false 的格子，然這兩個表示法對於變數的呈現方式也是相反的。總之，雖然這次在寫 verilog 的時候因為觀念錯誤而重新撰寫了幾次，但至少不是錯在隔一天的期中考中。